Project Overview

The goal of this project was to implement a game of Tetris, following the Super Rotation System guidelines for rotations and wallkicks. Specifically, we implemented the TetrisPiece and TetrisBoard classes to create the game logic. Additionally, we were also tasked with developing our own Tetris "Brain" to play the game and win at least 100 points.

Scope / Quality / Limitations

The scope of our project is the Super Rotation System mechanics applied to seven tetrominoes: the T, square, stick, left L, right L, left dog, and right dog, and each of their 4 different rotations. The project, especially the brain's algorithm, can be applied in the real world to logistics problems such as packing container ships or warehouses. The inputs are confined by the following actions: LEFT, RIGHT, DOWN, DROP, CLOCKWISE, COUNTERCLOCKWISE, NOTHING, and the dimensions of the Tetris board. The confines of the outputs are the results enumerated in the Board class: SUCCESS, OUT_BOUNDS, NO_PIECE, PLACE.

As for quality, we considered time complexity throughout the TetrisPiece implementation. In the constructor, we created a circular doubly linked list of rotations to ensure that all of the TetrisPiece accessor methods run in constant O(1) time. All of our TetrisBoard accessor methods also run in constant time. Additionally, we ensured quality of code by ensuring that our methods accounted for edge cases like wall kicks when applying rotations on the boundary and out-of-bounds exceptions.

Some limitations of our project include:

- Certain algorithms like testMove() are O(width x height), and thus the code won't be as efficient for larger boards.
- Traditional Tetris usually allows you to see the next piece, but our version doesn't.
- Since the game doesn't show the next piece beforehand. The brain is limited in the amount of information it has to make a decision.
- When iterating through all orientations in enumerateOptions, the brain wastes operations through 4 rotations of squares that give the same board since there's no written edge case for it.

- The weights were determined by guess and check and general assumptions, and could be optimized with a more solid testing approach.
- In the testing framework, MAX_MOVES and pieceCount in testBrainSurvivesManyPieces were arbitrarily set and thus could possibly not accurately represent the performance.

Algorithm & Implementation

We will begin by explaining our implementation of TetrisPiece. The constructor takes a PieceType object in its initial spawn orientation, with a body that is an array of Point objects. Each point is the lower-left corner of the piece's bounding box. The constructor then creates a circular doubly linked list of all four rotations of the piece. So, for each rotation, we stored a next and before variable of type TetrisPiece, where before refers to the counterclockwise rotation and next refers to the clockwise rotation. We did this by calling the helper method that we created, performClockwiseRotation(), four times. The performClockWiseRotation() method works by setting each x coordinate to the original y coordinate and each y coordinate to dimension - 1 - x-coordinate. In order to create the other three rotations, we made our own constructor that we can call that takes the type, rotationIndex, and body as arguments. Finally, we closed the circle by linking the third rotation to the spawn orientation. Next, our calculateSkirt() method works by creating an int array the size of the bounding box and filling it with default Integer.MAX_VALUE. Then, we iterate through and update it to the minimum height, or y-value, of each column before returning the array. That will ensure that a column with no blocks will remain Integer.MAX_VALUE. All of our accessor methods, including getType(), getRotationIndex(), clockwisePiece(), counterclockwisePiece(), getWidth(), getHeight(), getBody(), and getSkirt(), work by simply returning their respective instance variables that we declared at the beginning and calculated as explained above. Finally, our equals() method works by first ensuring that the other object being compared is an instance of TetrisPiece, then casting it to a TetrisPiece, and finally ensuring the piece type and orientation are the same.

Our TetrisBoard class maintains the state in a grid, which is a 2D array of PieceTypes, a current piece, and the current piece's position. The grid stores empty spots as null and occupied cells as PieceTypes. The move() method updates the result and resets the number of rows cleared during the action. It then calls the correct helper method that we created to perform the action through a series of switch cases. LEFT and RIGHT call our tryHorizontalShift() method, which checks whether the piece can move one column in the respective direction. DOWN calls tryVerticalShift(), which checks if a movement one row down is valid, and if not, then the piece is placed. DROP calls our drop() method, which keeps checking if the piece can move down

until isValid returns false and it's placed. CLOCKWISE and COUNTERCLOCKWISE both call our tryRotate() helper method, which tries to rotate the piece, applying wallkicks to keep the position valid when the rotation is at a wall. Finally, NOTHING just returns SUCCESS. For all of these, our isValid() helper method works by converting each piece's coordinates from relative to the bounding box position to actual grid coordinates, then ensuring that they're not out of bounds or overlapping with another block. For the rotations, we use a getPossibleKicks() helper method that returns an array of valid wall kicks a piece can perform given its type and rotation direction. getPossibleKicks() is useful as it generalizes the retrieval of wallkicks, therefore significantly reducing the amount of code as wall kicks are needed for all pieces other than square Our placePiece() method works by placing pieces by copying their blocks on the grid before updating the colHeights, rowWidth, and maxHeight variables and calling clearRows() to find and remove full rows. Our clearRows() method works by iterating through the rows, and when it finds a row that's not complete, it copies it over to the next available position and skips complete rows. Then, it replaces extra rows at the top with empty arrays and saves how many rows were cleared. If rows were removed, recalculateHeights() is called to recalculate the columnHeights and maximumHeight. Our testMove() method works by making a new TetrisBoard using the copy constructor, then applies the action to the copy, and returns the result. This is mainly only useful for the brain, so it can see what the result of some hypothetical actions would be without actually changing the board. Our accessor methods return the current piece position, dimensions, last action, last result, number of rows cleared, column heights, row widths, maximum height, and grid contents. Finally, our dropHeight() method uses the piece's skirt and columnHeight to find the placement position and safely drop without any overlap with the currently placed blocks.

Next, our JBrainTetris class extends JTetris and thus reuses the existing GUI, board, piece generation and logic. The main change is who makes the move, which is determined in the new constructor where a timer for the brain is created that calls tick() using Brain's nextMove() (more later) with a delay. This delay is specifically made to be less than the delay for the gravity drop so that the brain can react before a piece can fall (causing scores to be higher). The main method just creates the GUI using a JBrainTetris instance rather than the original JTetris.

Our custom brain, FirstBrain, largely takes from the original LameBrain, but makes a few crucial changes. The first change is that enumerateOptions() now enumerates the rotational orientations of every piece, thus giving the Brain more boards to work with. The second main change is with scoreBoard(). The score rewards rows cleared and penalizes total column height, holes (empty cells below a block), bumpiness (height differences between neighboring columns), deep wells, and a squared max-height term that makes the brain more careful as the stack grows.

These metrics come from Yiyuan Lee's Tetris AI write-up, however the weights that we determined are from trial and error, along with the Java implementation of obtaining the metrics. Finally, as LameBrain does, FirstBrain iterates through all options and scores them, choosing the board with the best score in nextMove().

Testing

We found testing TetrisBoard and TetrisPiece individually to be very beneficial. If we had to test the GUI of the system by validating each move, it would take an extremely long time and we likely would have missed many errors or edge cases, but JUnit made this very easy. Some examples of edge cases that were much easier to test with JUnit were multi-line collapses and kickbacks, which would have been difficult to test visually.

We tested our TetrisPiece class in the BlackBoxTetrisPieceTest file. We created a helper method bodySet to convert the point[] body into a set so the order of points doesn't matter. Some important tests we ran were ensuring the spawn piece orientation has rotationIndex 0; applying one clockwise (cw) rotation sets rotationIndex to 1; four rotations in the same direction return to the original; a clockwise rotation followed by counterclockwise returns to original; and verifying the second and third clockwise rotations. Additionally, we tested that the width and height are the same as that of the bounding box and remain the same after rotation; clockwise rotation for the T tetromino; verified the skirt computation for the T tetromino's 0 and 1 rotations, and finally we tested the equals method by ensuring that two TetrisPiece with the same type and orientation are equal.

We tested our TetrisBoard class in the BlackBoxTetrisBoardTest file. We created a createBoard helper method to create a new TetrisBoard object of dimensions 10x20. For these test cases, we ensured that a new board is constructed correctly by checking the dimensions and all the starting conditions. We tested the nextPiece() method by ensuring it correctly sets the current piece and its starting position. We created a lot of tests for the move() method, ensuring that it returns correctly when there is no active piece, left, right, or an attempt to move into a wall. We also tested that drop() works to move the block into the landing position. We also tested to ensure rows were cleared properly by dropping five square pieces into a 10-column board; the pieces should fill the bottom two rows, and they should be cleared out, so column height and maximum height should be updated, and the bottom row shouldn't have any remaining blocks. testRotationsAndWallKick() ensures that a piece shape T can correctly rotate clockwise and counterclockwise and their rotation index is correct. We also put a stick piece near the left wall, rotated it, moved it as far left as possible, and then tried to rotate counterclockwise to ensure that

the orientation is correct even when rotating a piece at a wall. Finally, we ran testEquals to ensure the equals() method correctly compares two boards with the same dimensions, piece type, and current piece position and returns true (they are equal).

The brain was tested using the TetrisTest class. First, constants are initialized representing the Tetris board's width and height, along with two setup methods. setUpAll() runs and stores every piece type, and setUpEach() creates a new FirstBrain instance before each test is run. testTetrisBoard() is largely taken from the Board's testing, just to make sure there aren't any errors before processing. The brain tests themselves have two helpers: spawn() that places a specific piece at the center top of the board, and playOut() that repeatedly has Brain generate a move until a piece is placed or it exceeds a max value. testBrainDoesNotChangeBoard() is pretty self-explanatory, it ensures that the brain's operations don't directly affect the original board. testBrainPlacesEveryPieceType() runs playOut() for all seven piece types to ensure the Brain is able to place all of them. Finally, testBrainSurvivesManyPieces() simulates a game with a fixed number of pieces and checks whether or not it can place all of them before losing the game. It uses a fixed random seed to allow us to reproduce the same conditions while slightly tweaking the weights.

Interesting Results

One interesting result was our first attempt at creating a brain. This FirstBrain implementation only scored based on minimizing maxHeight for each of the four rotations and their column drops, but the brain still did much better than LameBrain. Just adding the four rotations added a lot of flexibility and did a good job of keeping the stack of blocks low. However, since this first implementation of the Brain didn't penalize based on holes or surface bumpiness, it would sometimes trap open holes under blocks, which is what would make it eventually lose.

Another interesting result came from implementing checking the rotational orientations themselves. Intuitively, one would think to record a clockwise rotation with a left shift, both actions would have to be added to firstMoves simultaneously, but this isn't the case. By design, nextMove() can only return one Action, so it initially seemed like a mistake. If the best spot needs a rotation, it just rotates at that instant, rather than rotating beforehand and then moving towards it. This eliminates the need for an order of operation, and simplifies the implementation.

Assumptions

One important assumption I made during the testing for TetrisBoard is that the game board dimensions were set at a width of 10 blocks and a height of 20 blocks. I also assumed that

pieces always spawn at the middle of the top of the board and fall vertically down. I assumed that there wouldn't be any sort of attempt to spawn a piece when that spawning region is already occupied by blocks. We assumed that board dimensions are positive.

We also assumed that the seven Tetromino pieceTypes always follow the Super Rotation System (SRS) rules and specifications. For example, square pieces don't do wall kicks. We also assumed that there were no gravity rules that prevented floating blocks over holes.

For our brain, we assumed that JBrainTetris has a 10-millisecond brain delay, and it only moves when gameOn is true. So, we had to assume that this 10-millisecond timer was enough to search across all four rotations and columns. We assumed that the timer gives the brian enough time to evaluate all the options before the game drops and places it.

For the brain, we assumed that evaluating the possible rotations and placement positions was enough information to place a piece, especially because we didn't have the next piece that is going to be dropped visible.

Problems Encountered

One challenge we encountered was deciding how to store and access all four rotations in TetrisPiece. We knew we needed a data structure that would let us rotate between clockwise and counterclockwise rotations. At first, we considered using an array for this purpose, but we finally decided to use a circular doubly linked list to connect all four rotations with a next and before field. Each orientation stores references to the clockwise orientation and counterclockwise orientation, and the last orientation links back to the first. Overall, through this approach, we were able to ensure our accessor methods were all completed in constant time.

Another problem we encountered was ensuring that grid and colHeights, rowWidth, and maxHeight are always consistent. So, we had to make sure that every time a new piece is placed, the values in these variables are updated based on the changes that were made to which cells are occupied and which aren't. Similarly, when rows are cleared, the heights also change. So, we had to add in these recalculations.

Another problem we encountered was with JBrainTetris implementation. For our initial implementations, we couldn't figure out why the brain wasn't running. So, it took us a while to read through the entire implementation for JTetris and figure out which methods we had to override for the JBrainTetris to function properly and run our brain. Overall, this helped us learn and understand how inheritance and method overriding work.

Pair Programming

September 29 (Dinesh and Reva): Reading & understanding project spec, planning out TetrisPiece implementation, and started coding the first part. Reva drove for one hour, Dinesh drove for 1 hour.

September 29 (Dinesh): Finished implementation for circular doubly linked list of rotations for TetrisPiece, 1 hour

October 2 (Dinesh and Reva): Wrote test cases in BlackBoxTetrisPieceTest; Reva drove for one hour, Dinesh drove for two hours. Then, Reva drove debugging and commenting for half an hour.

October 6 (Dinesh and Reva): Began working on TetrisBoard; Dinesh drove for 1 hour, Reva drove for 1 hour.

October 8 (Dinesh): Finished implementation for TetrisBoard, 1 hour

October 9 (Reva): Wrote test cases for TetrisBoard, 1 hour

October 9 (Dinesh): Fixed error handling for TetrisBoard, half hour

October 14 (Dinesh and Reva): Wrote JBrainTetris and wrote FirstBrain. Dinesh drove for 2 hours, Reva drove for one hour.

October 15 (Dinesh): Fixed implementation for Brain, 1 hour

October 15 (Reva): Began the report, 2 hours

October 16 (Dinesh and Reva): Dinesh drove writing tests for 2 hours, Reva drove writing the report for 2 hours.

Social Impact

Since Tetris is inherently a user-facing game, accessibility should be considered when designing it because there are many potential users of our game who wouldn't be able to properly interact with it if they are differently-abled. Since our game relies so heavily on visual information, we could potentially be excluding players who have visual impairments, limited motor control, or other accessibility needs.

One improvement we could make is to provide other visual cues that don't rely exclusively on color. For example, Dinesh is colorblind, so while testing the game, he wasn't able to see the cyan blocks falling. A better implementation of the game could use

higher-contrast colors to accommodate colorblind users. Additionally, we could add patterns, symbols, or outlines to make pieces easier to distinguish.

Another improvement could be made for users with limited motor control by making it playable with other input methods. Currently, the Tetris game is played with keyboard controls to move, rotate, and drop pieces. However, we could make it playable with a mouse or controller to make it more accessible to players who have trouble using the keyboard for input.

We could also add optional sound for feedback, such as a sound effect when rows are cleared, a piece is moved or rotated, or the game is over. These audio cues would also help players by giving them more information about the game without relying solely on visual cues.

Overall, accessibility is very important when designing technology and software. It's important to keep the user's abilities in mind and find ways to ensure everyone can interact with our system fairly. By designing accessible technology, we can improve usability and overall improve the quality.